

INVESTIGATION REPORT

Issue 1007: Single-step with a stalled decode stage

openhwgroup/cv32e40p

Document	RTL-TRIAGE-CV32E40P-1007
Disposition	Dead code, proven
Record date	2026-09-10
Classification	Internal working document. Not a verification sign-off.

1 Status

Issue	openhwgroup/cv32e40p#1007
Issue filed	2024-06-18
Issue type	Coverage hole referred for disposition
Affects	cv32e40p
Configuration	COREV_PULP=1, FPU=0 (RTL master 6033d2b)
Disposition	Dead code, proven
Severity	Not assessed
Agreement	Same disposition as the independent human record.
Evidence	3 cited, 2 present, 1 not retained

Disposition meaning. The code cannot be reached in any state permitted by the stated model and assumptions, established by an unbounded proof.

Severity is not recorded for this run. The harness does not infer a severity from the disposition, so the field is reported as not assessed rather than filled in.

2 Description

cv32e40p_controller.sv:675 is the 'if (id_ready_i)' inside the DECODE single-step block (debug_single_step_i && !debug_mode_q, after the debug-entry and interrupt branches were not taken). The uncovered arm is id_ready_i==0. Proven unreachable with an unbounded abc pdr proof of the exact guard: whenever the controller is in DECODE with a valid instruction, single-step set and debug mode clear, id_ready_i is 1. Mechanism: dcsr.step can only be set while in debug mode, and the core leaves debug mode through dret -> FLUSH_EX -> FLUSH_WB -> DECODE with an empty EX/WB pipeline; the single-stepped instruction is therefore the first instruction after the flush, so no load-use, jalr, misaligned or CSR/APU hazard from a previous instruction can stall ID (id_ready = ~stalls & ex_ready, all sourced from older instructions), and the same cycle sends the FSM to DBG_FLUSH so no second instruction ever decodes under single-step. Anti-vacuity: the single-step block itself and its id_ready_i==1 arm are reached by the cover engine in 16 cycles, and DECODE with id_ready_i==0 is reached in 7 cycles (so the stall path exists, just not under single-step). The hole cover for id_ready_i==0 under single-step stayed unreached to depth 40 (cover job then timed out on that last cover, consistent with the proof).

3 Conditions

Not recorded as separate fields for this run. The conditions and environment assumptions that bound this result are stated inside the Description, and the configuration is given in Status.

4 Implications

Not recorded as a separate field for this run. What the disposition means for a user of the core follows from the Disposition meaning in Status and the bounds in Limitations and scope.

5 Action

Waiver text, to be applied to the coverage exclusion

cv32e40p_controller.sv:675 'if (id_ready_i)' under 'debug_single_step_i & ~debug_mode_q' in DECODE: the AllFalse arm (id_ready_i==0) is formally unreachable (abc pdr unbounded proof, runs/v1-cv32e40p-1007/formal/p_line675). Single-step is only active for the first instruction after leaving debug mode, when EX/WB are empty, so no stall source exists; the same cycle moves the FSM to DBG_FLUSH. The enclosing block is reachable (cover step 16), so the proof is not vacuous. Exclude the id_ready_i==0 arm from branch coverage (open-source proof replaces the Cadence SLEC equivalence argument).

Checker used

checker_1007.sv

6 Evidence

Ref	Type	Path, relative to the run directory	File	Result recorded
E1	proof	formal/p_line675/sby.log	present	abc pdr PASS unbounded: DECODE && instr_valid && !dbg-entry && !irq && debug_single_step_i && !debug_mode_q -> id_ready_i
E2	proof	formal/cover/sby.log	present	cover: single-step block reached at step 16, its id_ready arm at step 16, DECODE&&!id_ready at step 7 (3 of 4 covers); the hole cover unreached to depth 40 (job TIMEOUT on that cover only)
E3	counterexample	formal/cover/job/engine_0/trace1.vcd	not kept	witness of the single-step block being entered (rendered in cover_single_step_block_reached.png)

Paths are relative to runs/v1-cv32e40p-1007. The File column is a check made when this document was generated. It reports whether the cited file exists, not whether its contents were re-validated. 'not kept' means a repository rule deliberately excludes that artefact, usually a waveform dump; rerun the check to regenerate it. 'MISSING' means the file is absent and was expected to be there.

What each class of artefact establishes

- proof. Proof or engine run. Scope depends on the mode: prove is unbounded under the stated model, bmc is bounded to its depth, cover establishes reachability only.
- counterexample. A concrete input sequence produced by the engine. Shows the stated condition is reachable; it is not itself a proof.

Artefact(s) E3 are not retained under repository policy. The result recorded against them stands on the engine log cited alongside; the artefact itself must be regenerated by rerunning the check before it can be inspected.

7 Limitations and scope

- The proof establishes unreachability under the model and the environment assumptions named in Status and Conditions. It does not establish that the surrounding logic is correct.
- This is an internal investigation record. It requires engineering review before it is used as a disposition. No report generator can supply missing evidence.
- Results apply to the configuration and core revision named in Status. A different configuration needs a separate run.
- Public demonstration runs on open cores are not client engagements and not production verification results.

8 Provenance and measurement

Run directory	runs/v1-cv32e40p-1007
Title source	Title and configuration read back from reports/issues/cv32e40p_issue_1007.pdf, the previously published report for this issue.
Inputs shown	title, body(links stripped), labels
Human record	Dolphin Design 2024 (issue body): waived in v2 after Cadence JasperGold SLEC showed removing lines 675/703 leaves outputs unchanged

Assessed confidence	0.9. This is the investigator's own estimate. It is not a measurement and not a reliability figure.
Iterations	1

Generated by reporting/scripts/issue_document.py from runs/v1-cv32e40p-1007/verdict.json. The generator typesets recorded data and checks that cited files exist. It does not run, rerun or validate any check, and it cannot supply missing evidence.